Practica 4 TALF

José Ruiz Pareja

2 de junio de 2026

1. Implementa un programa WHILE que calcule la suma de tres valores

He definido el siguiente programa WHILE:

```
Q: (3, 4, s)
s:
X4 := 0;

while X1 != 0 do
    X4 := X4 + 1;
    X1 := X1 - 1
od;

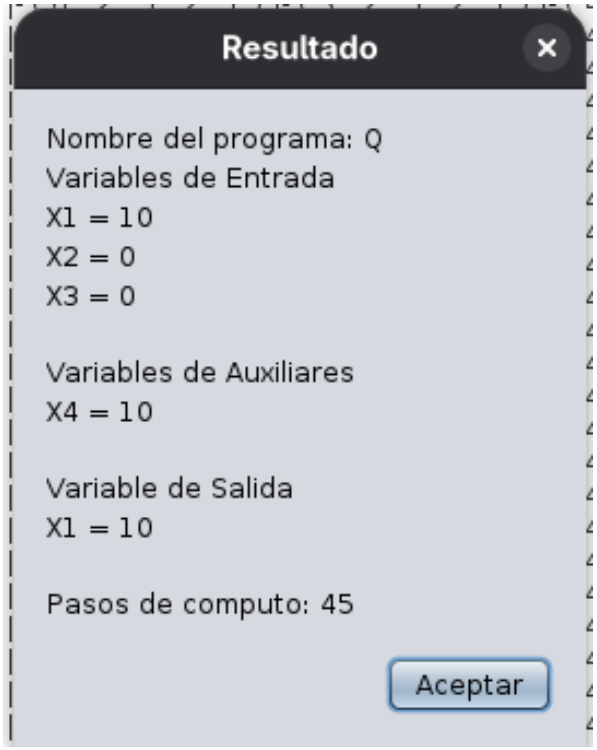
while X2 != 0 do
    X4 := X4 + 1;
    X2 := X2 - 1
od;

while X3 != 0 do
    X4 := X4 + 1;
    X3 := X3 - 1
od;

X1 := X4
```

1.1. Ejecútala con el compilador de WHILE y halla $T_Q(3, 5, 2)$

Donde usamos X4 como variable auxiliar para acumular el resultado de la suma. Si cargamos el programa e introducimos las variables (3,5,2) podemos ver el resultado de la suma y T_Q(3,5,2).



Como podemos ver el resultado de la suma es el valor asignado a la variable X1 (10), y

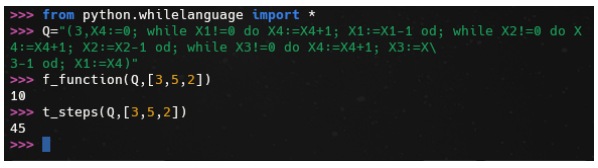
$$T_Q(3, 5, 2) = 45$$

1.2. Ejecútala en Python y comprueba que coincide.

Primero definimos el programa de la siguiente forma:

```
Q="(3,X4:=0; while X1!=0 do X4:=X4+1; X1:=X1-1 od; while X2!=0 do X4:=X4+1;
X2:=X2-1 od; while X3!=0 do X4:=X4+1; X3:=X3-1 od; X1:=X4)"
```

Y ejecutamos las funciones $f_function(Q, [3, 5, 2])$ y $t_steps(Q, [3, 5, 2])$ para averiguar el valor resultante y la complejidad temporal del programa con la entrada (3,5,2)



2. Crea el programa WHILE más sencillo que calcule la función diverge (con cero argumentos) y calcula la codificación de su código.

Primero definimos el siguiente programa WHILE:

```
P=(0, 1, s)
s:
X1:=X1 + 1
while X1!=0 do
    X1:=X1
od
```

Que sería un bucle infinito, por lo que divergería siempre ya que no tiene variables de entrada. Usamos la funcion $while2n$ en python para averiguar la codificacion del programa WHILE

```
>>> while2n(0,"X1:=X1+1; while X1!=0 do X1:=X1 od")
9678230627
>>>
```

Como podemos ver la codificacion del codigo es 9678230627

3. Crea una función de Python que enumere los k primeros vectores de N*

La funcion definida es:

```
from .godeldecoding import godeldecoding

def enumN(n: int) -> int:
    res=[]
    for i in range(n):
        res.append(godeldecoding(i))
    return res
```

Que si la probamos en terminal vemos que funciona:

```
>>> enumN(8)
[[], [0], [0, 0], [1], [0, 0, 0], [1, 0], [2], [0, 0, 0, 0]]
>>> enumN(0)
[]
>>> enumN(5)
[[], [0], [0, 0], [1], [0, 0, 0]]
>>>
```

4. Crea una función de Python que enumere los k primeros programas WHILE.

La funcion definida es:

```
from .n2while import n2while


def enumWhile(n: int) -> str:
    res=[]
    for i in range(n):
        res.append(n2while(i))
    return res
```

Que si la probamos en terminal vemos que funciona:

